

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:

Я.В. Автушенко

Студент группы ПО-13

Проверил:

А.Д. Кулик

преп.-стажёр кафедры ИИТ

Цель: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1:

Реализовать простой класс

Требования к выполнению

- Реализовать пользовательский класс по варианту.

Для каждого класса

- Создать атрибуты (поля) классов
- Создать методы классов
- Добавить необходимые свойства и сеттеры (по необходимости)
- Переопределить магические методы `__str__` и `__eq__`

1) Равнобедренный треугольник, заданный длинами сторон – Предусмотреть возможность определения площади и периметра, а также логический метод, определяющий существует ли такой треугольник. Конструктор должен позволять создавать объекты с начальной инициализацией. Переопределить метод `__eq__`, выполняющий сравнение объектов данного типа.

`import math`

```
class IsoscelesTriangle:
```

```
    def __init__(self, a, b, c):
```

```
        if a <= 0 or b <= 0 or c <= 0:
```

```
            raise ValueError("Стороны должны быть положительными")
```

```
        if not (math.isclose(a, b) or math.isclose(a, c) or math.isclose(b, c)):
```

```
            raise ValueError("Треугольник должен быть равнобедренным")
```

```
        if not (a + b > c and a + c > b and b + c > a):
```

```
            raise ValueError("Треугольник не существует")
```

```
        self.a, self.b, self.c = a, b, c
```

```
    def exists(self):
```

```
        return True
```

```
    def perimeter(self):
```

```
        return self.a + self.b + self.c
```

```
    def area(self):
```

```
        p = self.perimeter() / 2
```

```
        return math.sqrt(p * (p - self.a) * (p - self.b) * (p - self.c))
```

```
    def __str__(self):
```

```
        return (f"Равнобедренный треугольник: {self.a:.2f}, {self.b:.2f}, {self.c:.2f}\n"
```

```
                f"Периметр: {self.perimeter():.2f}, Площадь: {self.area():.2f}")
```

```
    def __eq__(self, other):
```

```
        if not isinstance(other, IsoscelesTriangle):
```

```
            return False
```

```
        return sorted([self.a, self.b, self.c]) == sorted([other.a, other.b, other.c])
```

```

# Демонстрация
if __name__ == "__main__":
    t1 = IsoscelesTriangle(5, 5, 3)
    t2 = IsoscelesTriangle(3, 5, 5)
    t3 = IsoscelesTriangle(6, 6, 6)

    print("Треугольник 1:")
    print(t1)
    print(f"Существует: {t1.exists()}\n")

    print("Треугольник 2:")
    print(t2)
    print(f"Существует: {t2.exists()}\n")

    print(f"Треугольник 1 == Треугольник 2: {t1 == t2}")
    print(f"Треугольник 1 == Треугольник 3: {t1 == t3}")

    # Проверка ошибок
    try:
        IsoscelesTriangle(3, 4, 5) # Не равнобедренный
    except ValueError as e:
        print(f"\nОшибка: {e}")

    try:
        IsoscelesTriangle(1, 1, 3) # Не существует
    except ValueError as e:
        print(f"Ошибка: {e}")

```

```

Треугольник 1:
Равнобедренный треугольник: 5.00, 5.00, 3.00
Периметр: 13.00, Площадь: 7.15
Существует: True

Треугольник 2:
Равнобедренный треугольник: 3.00, 5.00, 5.00
Периметр: 13.00, Площадь: 7.15
Существует: True

Треугольник 1 == Треугольник 2: True
Треугольник 1 == Треугольник 3: False

Ошибка: Треугольник должен быть равнобедренным
Ошибка: Треугольник не существует
Для продолжения нажмите любую клавишу . . .

```

Задание 2:

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной

системы

1) Система Факультатив. Преподаватель объявляет запись на Курс. Студент записывается на Курс, обучается и по окончании Преподаватель выставляет Оценку, которая сохраняется в Архиве. Студентов, Преподавателей и Курсов при обучении может быть несколько.

```
from abc import ABC, abstractmethod
```

```
from datetime import datetime
```

```
from enum import Enum
```

```
from typing import List, Dict, Optional
```

```
class Grade(Enum):
```

```
    A = 5; B = 4; C = 3; D = 2; F = 1
```

```
    def __str__(self): return f'{self.name}({self.value})'
```

```
class Person(ABC):
```

```
    def __init__(self, name: str):
```

```
        self.name = name
```

```
        self.id = id(self)
```

```
    @abstractmethod
```

```
    def get_role(self): pass
```

```
    def __str__(self): return f'{self.get_role(): {self.name}]'
```

```
class Student(Person):
```

```
    def __init__(self, name: str):
```

```
        super().__init__(name)
```

```
        self.enrollments: List['Enrollment'] = []
```

```
    def get_role(self): return "Студент"
```

```
    def enroll(self, course: 'Course') -> 'Enrollment':
```

```
        if any(e.course == course for e in self.enrollments):
```

```
            raise ValueError(f'{self.name} уже записан на {course.title}')
```

```
        enrollment = Enrollment(self, course)
```

```
self.enrollments.append(enrollment)

course.add_enrollment(enrollment)

return enrollment
```

```
class Teacher(Person):
    def __init__(self, name: str):
        super().__init__(name)
        self.courses: List['Course'] = []
    def get_role(self): return "Преподаватель"
    def create_course(self, title: str, max_students: int = 30) -> 'Course':
        course = Course(title, self, max_students)
        self.courses.append(course)
        return course
    def assign_grade(self, enrollment: 'Enrollment', grade: Grade):
        if enrollment.course.teacher != self:
            raise ValueError("Не ваш курс")
        enrollment.set_grade(grade)
```

```
class Course:
    def __init__(self, title: str, teacher: Teacher, max_students: int = 30):
        self.title = title
        self.teacher = teacher
        self.max_students = max_students
        self.enrollments: List['Enrollment'] = []
        self.status = "Открыт"
        self.id = id(self)
    def add_enrollment(self, enrollment: 'Enrollment'):
        if len(self.enrollments) >= self.max_students:
            raise ValueError(f"Курс {self.title} заполнен")
        self.enrollments.append(enrollment)
    def start(self): self.status = "Идет"
    def complete(self): self.status = "Завершен"
```

```

    def __str__(self): return f"Курс: {self.title} ({self.status}, {len(self.enrollments)}/{self.max_students})"

```

```

class Enrollment:

```

```

    def __init__(self, student: Student, course: Course):
        self.student = student
        self.course = course
        self.date = datetime.now()
        self.grade: Optional[Grade] = None

    def set_grade(self, grade: Grade):
        self.grade = grade
        Archive().add_record(self)

    def __str__(self): return f"{self.student.name} -> {self.course.title}: {self.grade or 'Нет оценки'}"

```

```

class Archive:

```

```

    _instance = None

    def __new__(cls):
        if not cls._instance:
            cls._instance = super().__new__(cls)
            cls._instance.records: List[Enrollment] = []
        return cls._instance

    def add_record(self, enrollment: Enrollment):
        self.records.append(enrollment)

    def get_student_grades(self, student: Student) -> List[Enrollment]:
        return [r for r in self.records if r.student == student]

    def __str__(self): return f"Архив: {len(self.records)} записей"

```

```

# Демонстрация

```

```

if __name__ == "__main__":
    # Создание преподавателей и студентов
    t1 = Teacher("Иванов И.И.")

```

```
t2 = Teacher("Петрова А.С.")
```

```
s1 = Student("Смирнов А.")
```

```
s2 = Student("Козлова М.")
```

```
s3 = Student("Новиков Д.")
```

```
# Создание курсов
```

```
c1 = t1.create_course("Математический анализ", 2)
```

```
c2 = t2.create_course("Python для начинающих", 2)
```

```
print("==== Преподаватели объявляют курсы ====")
```

```
print(c1, c2, sep="\n")
```

```
# Запись на курсы
```

```
print("\n==== Студенты записываются ====")
```

```
e1 = s1.enroll(c1)
```

```
e2 = s2.enroll(c1)
```

```
e3 = s1.enroll(c2)
```

```
e4 = s3.enroll(c2)
```

```
# Попытка превысить лимит
```

```
try:
```

```
    s3.enroll(c1)
```

```
except ValueError as e:
```

```
    print(f'Ошибка: {e}')
```

```
print(e1, e2, e3, e4, sep="\n")
```

```
# Обучение
```

```
print("\n==== Обучение ====")
```

```
c1.start()
```

```
c2.start()
```

```
print(f"{c1.title}: {c1.status}")
print(f"{c2.title}: {c2.status}")
```

```
# Завершение и оценки
```

```
print("\n=== Завершение и оценки ===")
```

```
c1.complete()
```

```
c2.complete()
```

```
t1.assign_grade(e1, Grade.A)
```

```
t1.assign_grade(e2, Grade.B)
```

```
t2.assign_grade(e3, Grade.A)
```

```
t2.assign_grade(e4, Grade.C)
```

```
# Архив
```

```
print("\n=== Архив ===")
```

```
archive = Archive()
```

```
print(archive)
```

```
print(f"\nОценки {s1.name}:")
```

```
for e in archive.get_student_grades(s1):
```

```
    print(f" {e}")
```

```
print(f"\nОценки {s2.name}:")
```

```
for e in archive.get_student_grades(s2):
```

```
    print(f" {e}")
```



```
=== Преподаватели объявляют курсы ===
Курс: Математический анализ (Открыт, 0/2)
Курс: Python для начинающих (Открыт, 0/2)

=== Студенты записываются ===
Ошибка: Курс Математический анализ заполнен
Смирнов А. -> Математический анализ: Нет оценки
Козлова М. -> Математический анализ: Нет оценки
Смирнов А. -> Python для начинающих: Нет оценки
Новиков Д. -> Python для начинающих: Нет оценки

=== Обучение ===
Математический анализ: Идет
Python для начинающих: Идет

=== Завершение и оценки ===

=== Архив ===
Архив: 4 записей

Оценки Смирнов А.:
  Смирнов А. -> Математический анализ: A(5)
  Смирнов А. -> Python для начинающих: A(5)

Оценки Козлова М.:
  Козлова М. -> Математический анализ: B(4)
Для продолжения нажмите любую клавишу . . .
```

Вывод: были отработаны базовые знания языка программирования Python при решении практических задач